LERO Software for a better world

A World Leading SFI Research Centre

Science Foundation Ireland sfi For what's next

CS5741
Concurrency & Parallelism in Software Development

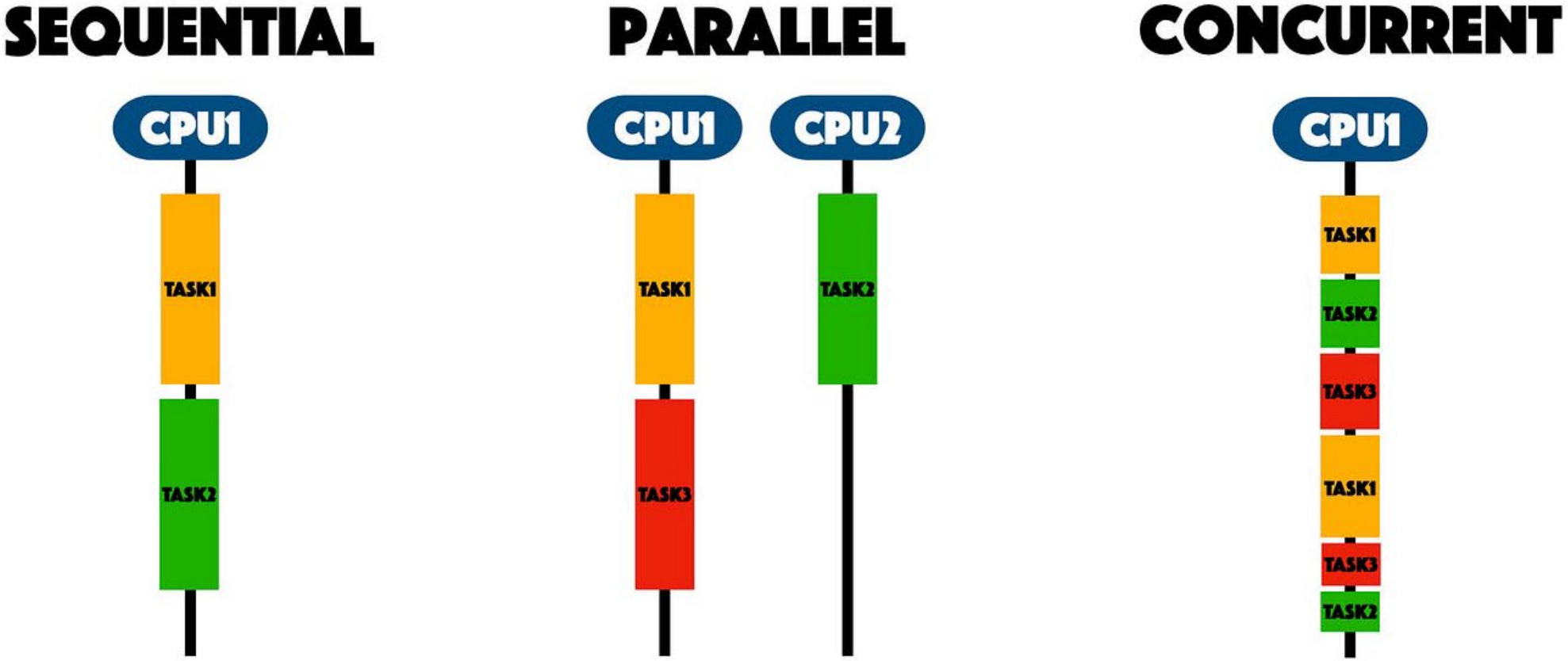
Lecture 2 – Identifying Parallelisable Problems

Dr Faeq Alrimawi

HOST INSTITUTION
UNIVERSITY OF LIMERICK OLLSCOIL LUIMNIGH

PARTNER INSTITUTIONS
NUI Galway OÉ Gaillimh · Maynooth University · Trinity College Dublin, Coláiste na Tríonóide, Baile Átha Cliath, The University of Dublin · University College Dublin, An Coláiste Ollscoile, Baile Átha Cliath · UCC · MTU, Ollscoil Teicneolaíochta na Mumhan, Munster Technological University · DCU · Waterford Institute of Technology · Dundalk Institute of Technology, Dún Dealgan · LIT · GMIT

Recap (1)… Terminology



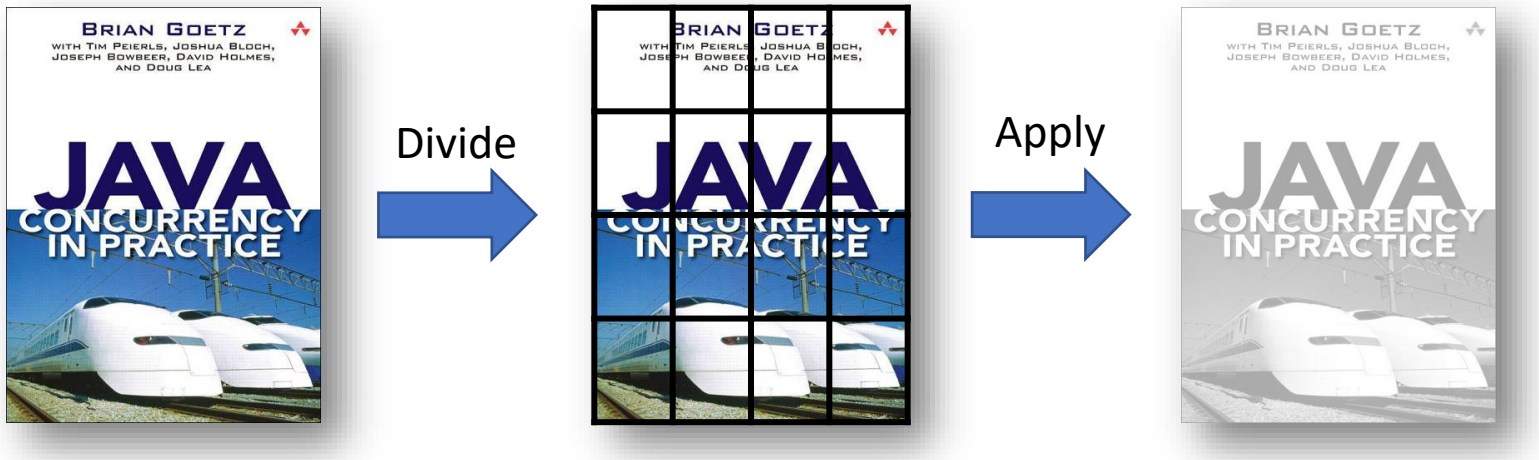
Today, We Are Covering…

- What Are Parallelisable Problems?
- Their Characterisations
- Decomposition Analysis
- Data Parallelism vs. Task Parallelism
- Amdahl's Law and Scalability

What Are Parallelisable Problems?

- Those that can be **divided into smaller, independent tasks** that can be **executed simultaneously**
  - Sub-tasks do not interfere with each other
  - Allows for parallel execution across multiple processors or cores

- Example: Applying filter (e.g., greyscale) to an image



Some Problems Are Greatly Parallelisable

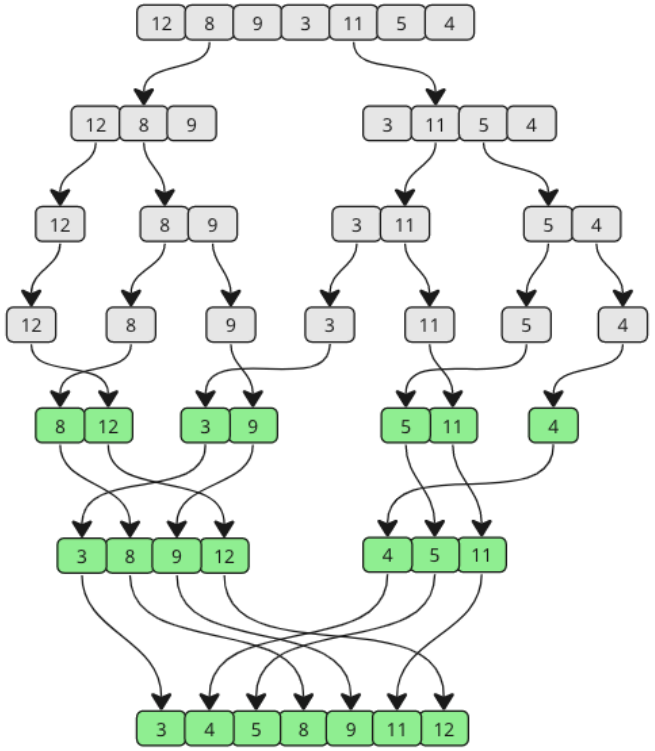
- For example finding the sum of integers in an array

```
for (int value : array) {
    sum += value;
}
```



Some Problems Are Partially Parallelisable

- For example, the sorting algorithm **Merge Sort**



Some Problems Are NOT Parallelisable

- For example, finding the Fibonacci Sequence of n

Fibonacci Sequence

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987 ...

Each number is the sum of the previous two numbers.

```
int fibonacci(int n) {
    if (n <= 1) {
        return n;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

WHY?

Today, We Are Covering…

- What Are Parallelisable Problems?
- Their Characterisations
- Decomposition Analysis
- Data Parallelism vs. Task Parallelism
- Amdahl's Law and Scalability

Parallelisable Problem Characteristics

- Decomposability
  - The problem can be **divided** into **smaller chunks** (sub-problems).
  - Example: Matrix multiplication where each element of the result matrix can be computed separately.
- Independence
  - Tasks should be independent or have **minimal dependencies**.
  - Example: Image processing tasks where each pixel can be processed independently.
- Granularity
  - The **size** of the tasks should be **large/small enough** to **justify** the **overhead** of parallel execution.
  - Example: Sorting large arrays versus small arrays.

Parallelisable Problem Characteristics: Decomposability

- The ability of a problem to be broken down into smaller, **manageable sub-problems** that can be solved concurrently or in parallel.

- For parallelism, a problem should be decomposable into tasks that can be executed in parallel with **no or minimal dependencies**

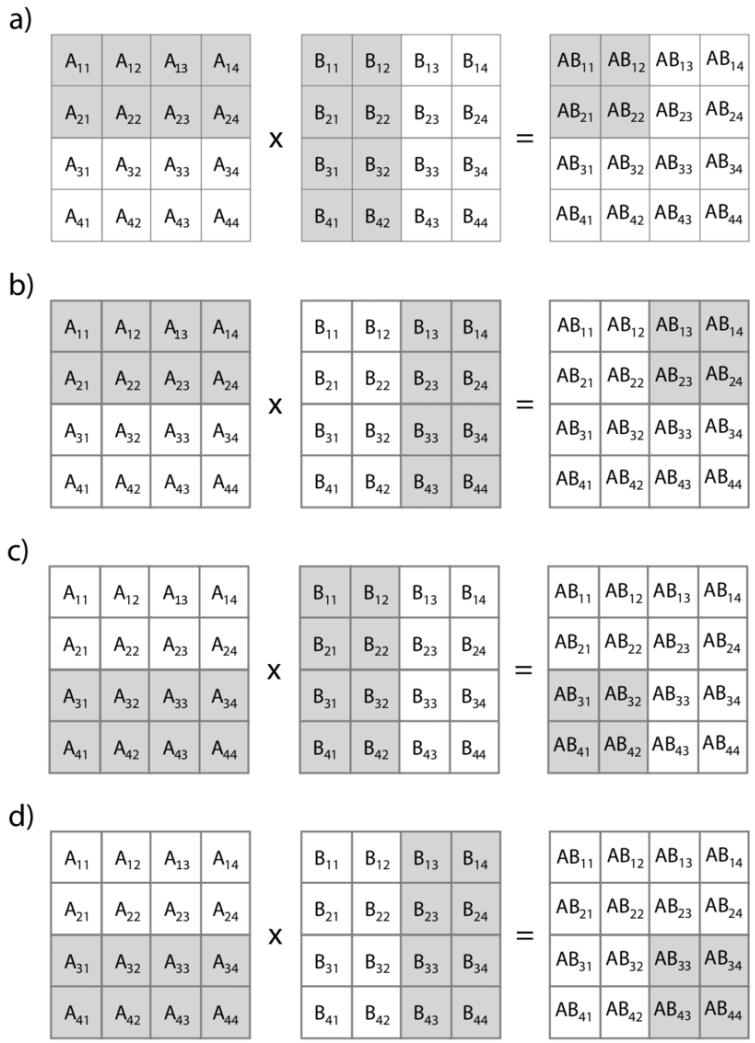
Decomposable Problems Characteristics

- **Modularity**: The problem can be modularised into smaller sub-tasks or data chunks.
- **Scalability**: Decomposability allows for scaling up by adding more processors or threads.
- **Granularity Control**: Decomposition allows fine control over task granularity to optimise performance.

Challenges to Decomposability

- **Dependency Management**: Ensuring that decomposed **tasks do not interfere with each other** or cause conflicts.
- **Load Balancing**: Ensuring that each decomposed task has a roughly **equal amount of work** to avoid bottlenecks.

Example: Matrix Multiplication



src: https://www.researchgate.net/figure/One-possible-blocked-decomposition-of-the-matrix-multiplication-of-two-4-4-matrices_fig21_325076288

Parallelisable Problem Characteristics: Independence

- The degree to which sub-tasks or sub-problems can **operate without relying on each other**.

- For parallelism, **tasks** should **ideally** be **independent**
  - meaning that the completion of one task does not depend on the results of another.

Types of Independence

- Data Independence
  - Tasks operate on different pieces of data, so changes in one task do not affect others.
  - Example: Applying sum task to different sections of an array.

- Task Independence
  - Tasks do not need to communicate or synchronise with each other.
  - Example: Performing multiple independent analyses on a document, such as keyword search, word count, and sentiment analysis.

Independent Problems Characteristics

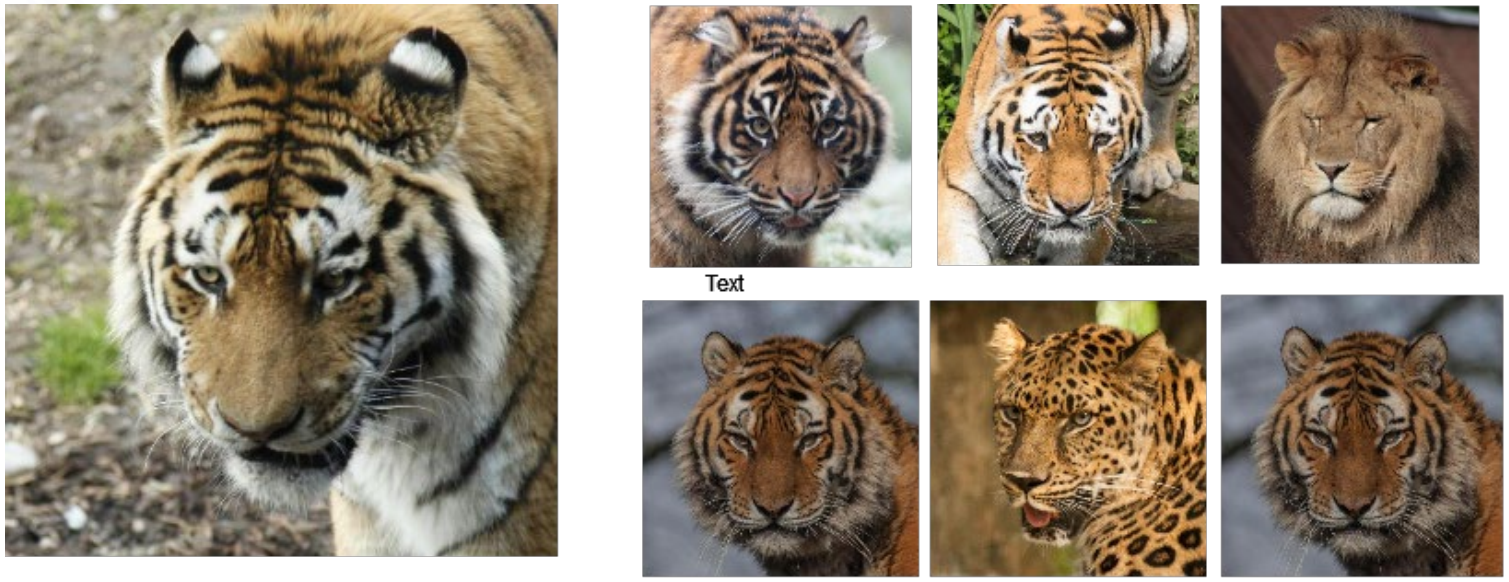
- **Reduced Synchronisation**: Less need for synchronisation mechanisms, which minimises overhead and contention.
- **Parallel Efficiency**: High potential for effective parallelism as tasks do not block each other.
- **Error Isolation**: Errors in one task do not propagate to others.

Challenges

- **Coordination Overhead**: Even independent tasks may require some level of coordination or merging of results.
- **Dependency Resolution**: Ensuring that dependencies are correctly managed if tasks are not entirely independent.

Example: Search a Large Dataset

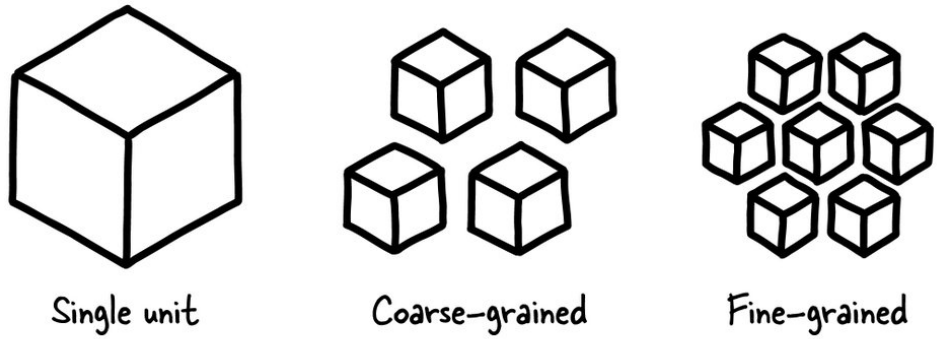
- Searching different portions of a large dataset in parallel, where each search is independent of the others
  - E.g., searching for similar images in a database



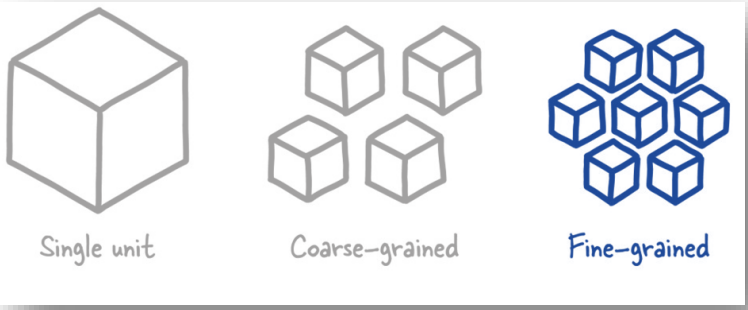
Parallelisable Problem Characteristics: Granularity

- The size or amount of work performed by each parallel task

- Can be:
  - **Fine-Grained**: dividing a problem into **many small tasks**, each of which performs a relatively small amount of computation.
  - **Coarse-Grained**: dividing a problem into **fewer**, **larger tasks**, each of which performs a significant amount of computation.



Fine-Grained Characteristics



- **Large Number of Small Tasks**: breaking down a problem into a large number of small tasks.
  - Each task performs a minimal amount of computation.
- **Frequent Communication**: tasks MAY often need to communicate or synchronise with each other frequently.
  - This can introduce significant overhead due to the cost of communication and synchronisation, potentially negating the benefits of parallelism.
- **Higher Overhead**: the overhead of managing a large number of small tasks can be high.
  - For example, in a multi-threaded environment, creating, scheduling, and switching between threads incurs overhead that may outweigh the benefits of parallelism if the tasks are too fine-grained.
- **High Scalability Potential**: fine-grained parallelism can scale well if the underlying system can efficiently manage the large number of tasks.
  - However, on systems with limited resources, the overhead can lead to poor performance.

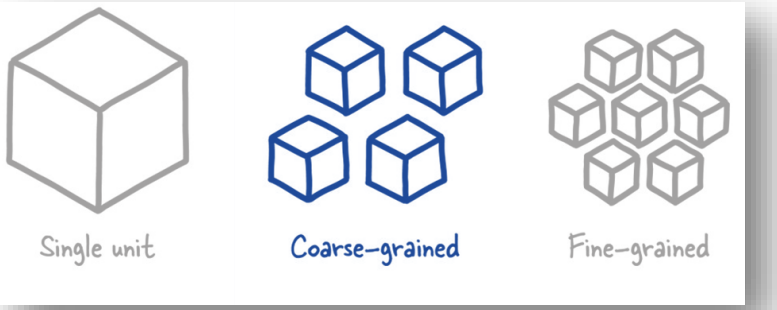
Fine-Grained Example

- Matrix Addition
  - Each element of the result matrix is computed independently.
  - For a matrix of size 1000×1000, there will be 1,000,000 tasks.

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix} \text{ and } B = \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix}$$

$$A + B = \begin{bmatrix} a_{11} + b_{11} & a_{12} + b_{12} & a_{13} + b_{13} \\ a_{21} + b_{21} & a_{22} + b_{22} & a_{23} + b_{23} \\ a_{31} + b_{31} & a_{32} + b_{32} & a_{33} + b_{33} \end{bmatrix}$$

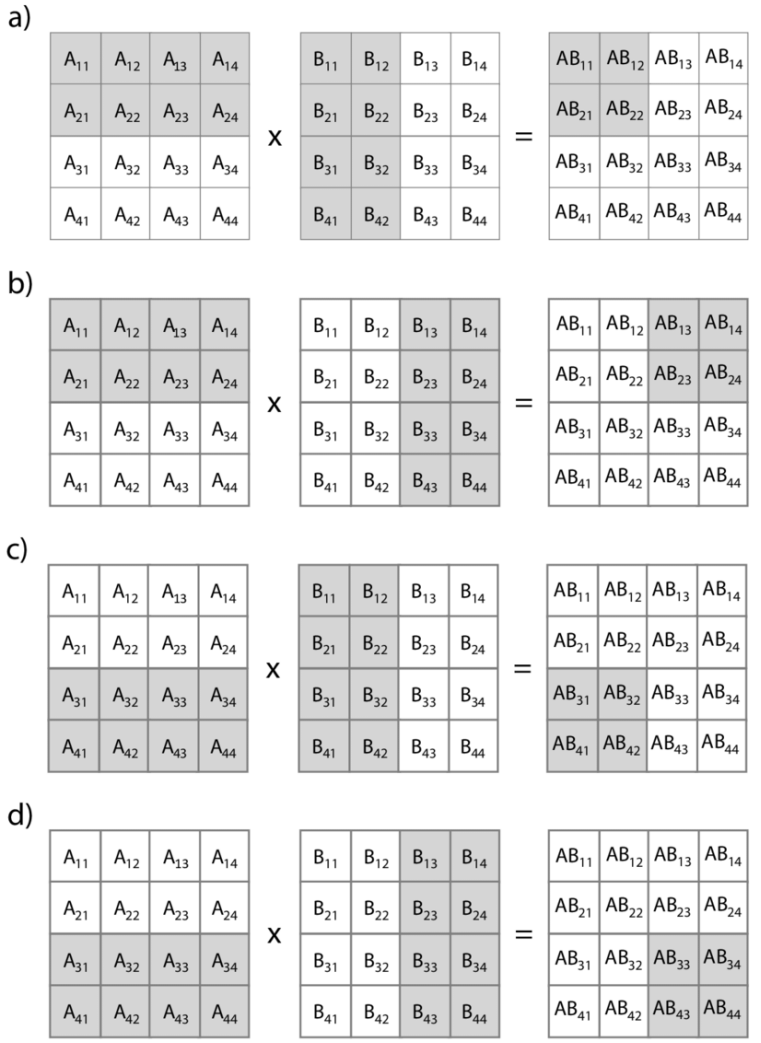
Coarse-Grained Characteristics



- **Small Number of Large Tasks**: breaking down a problem into a small number of large tasks.
  - Each task performs a maximal amount of computation.
- **Infrequent Communication**: larger tasks generally require less frequent communication or synchronisation.
  - This can lead to better performance, especially in distributed systems where communication cost is high.
- **Lower Overhead**: managing fewer tasks means lower overhead in terms of task scheduling, communication, and synchronisation.
  - This makes coarse-grained parallelism more efficient in environments where task management overhead is a concern.
- **Low Scalability Potential**: Coarse-grained parallelism may not scale as well as fine-grained parallelism because there are fewer tasks to distribute across multiple processors.

Coarse-Grained Example

- Matrix Multiplication
  - The problem is divided into larger tasks, where each task might involve computing an entire row or a block of rows of the resulting matrix
  - For example, if you have **four processors**, the matrix multiplication task could be divided into **four sub-tasks**, each responsible for calculating a quarter of the resulting matrix



Fine Or Coarse? Balancing Granularity

- The key **challenge** in parallel computing is finding the **right balance** between fine-grained and coarse-grained parallelism.
  - **too fine a granularity** can lead to excessive **overhead**, while **too coarse a granularity** can result in **underutilisation** of available parallel resources.
- The **optimal granularity** depends on several **factors**, including:
  - Nature of the problem
  - Architecture of the system
  - Communication costs
- **Adaptive Granularity**: some parallel algorithms and frameworks can **dynamically adjust granularity** based on runtime conditions.

Techniques for Identifying Parallelisable Problems

Today, We Are Covering…

- What Are Parallelisable Problems?
- Their Characterisations
- **Decomposition Analysis**
- Data Parallelism vs. Task Parallelism
- Amdahl's Law and Scalability

Decomposition Analysis

- Breaking down a problem into smaller, manageable sub-problems that can be executed concurrently or in parallel.

- Key Steps:
  - **Components Analysis**: determine the major components of the problem.
  - **Dependency Analysis**: ensure tasks are independent or have minimal dependencies.
  - **Communication Analysis**: analyse the communication requirements between tasks

Component Analysis

- **Identify** and define the **individual components (tasks)** of the problem that can be executed independently.

- Steps:
  - Break down the problem into **distinct, smaller tasks** or components.
  - Ensure that **each component** is **self-contained**, meaning it can be executed independently of others to a certain extent.

- Example: matrix addition, each element of the result matrix can be computed independently.

Dependency Analysis

- Identify dependencies between components to **determine the sequence of execution** and whether components can be executed in parallel.

- Steps:
  - Analyse which **components rely** on the results of **others**.
  - Determine the **execution order** based on these dependencies.

- Example: data processing pipeline, fetch data (task A), apply filter1 (task B), apply filter2 (task C).
  - Task B and C depend on the result of task A, while Task B and C are independent

Communication Analysis

- The communication requirements between components
  - Such as data to send or receive

- Steps:
  - Determine how components will **exchange data** or **synchronise** their operations.
  - Analyse the **volume** of data that needs to be communicated, the **frequency** of communication, and the **method** (e.g., message-passing, shared memory).

- Example: matrix multiplication in distributed systems, task with partial matrix needs to access the main matrix, it also needs to communicate the results of multiplication back to the main processor.

Decomposition Analysis: Example

- Summing a Large Array

1	2	3	4	5	6	7	…	10M

- Component Analysis: identify subtasks
- Dependency Analysis: determine the sequence of execution
- Communication Analysis: check if any data need to be exchanged

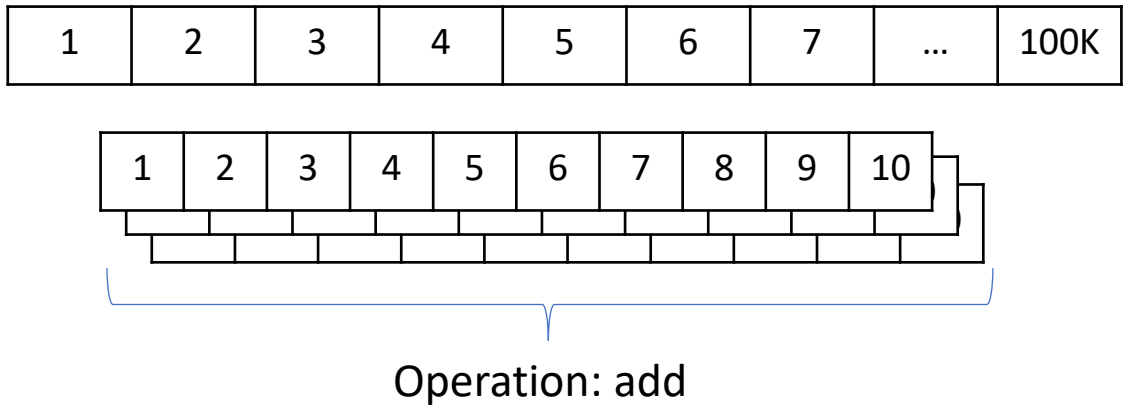
Today, We Are Covering...

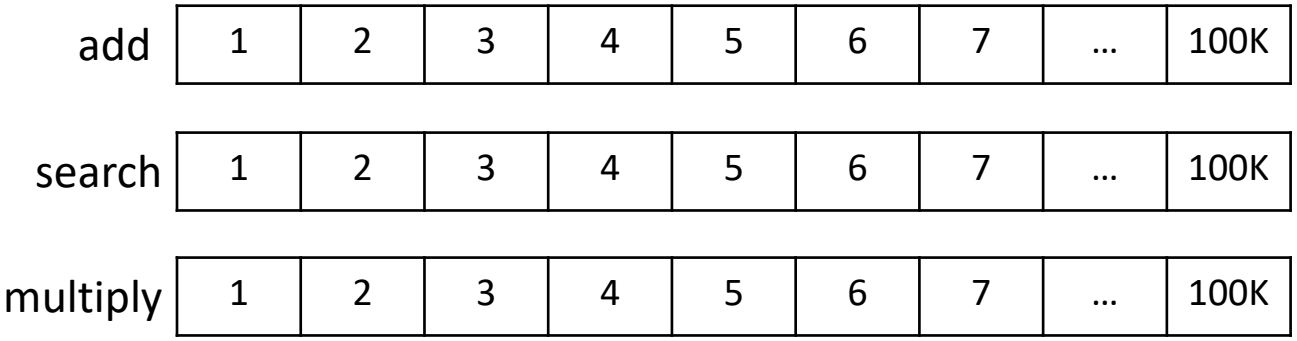
- What Are Parallelisable Problems?
- Their Characterisations
- Decomposition Analysis
- Data Parallelism vs. Task Parallelism
- Amdahl's Law and Scalability

Data Parallelism vs. Task Parallelism

- Two paradigms to divide the problem.

- **Data parallelism** focuses on **dividing** the **data**
- **Task parallelism** focuses on **dividing** the **tasks** (functions/operations)





Data Parallelism

- Focuses on distributing data across multiple processing units.
  - Each processing unit performs the same operation on different pieces of the data simultaneously.
- Characteristics:
  - **Uniform Operations**: the **same operation** is applied to **all data** elements, often in parallel.
  - **Scalability**: typically **scales well** with the amount of data, as more processing units can be added to handle larger datasets.
  - **Independence**: each data element or **chunk** of data is **processed independently** of others, reducing the need for inter-process communication.
- Example: finding the sum of an array.

Data Parallelism

Advantages

- **Efficiency**: Ideal for operations on large datasets where the same computation is performed repeatedly.
- **Simplicity**: Easier to implement for problems that naturally fit this model.
- **Performance**: Can achieve significant speedup for data-intensive tasks.

Disadvantages

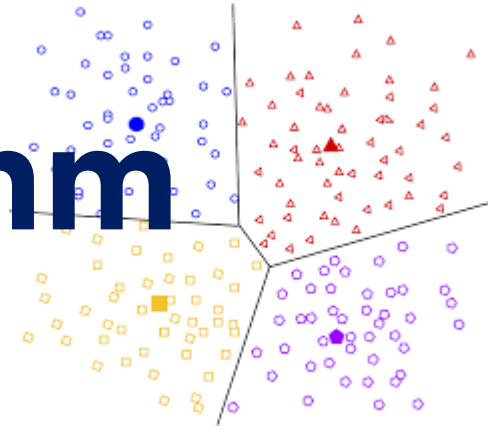
- **Limited to Certain Problems**: Not all problems can be easily expressed in terms of data parallelism.
- **Synchronisation Overhead**: While data elements are processed independently, combining results may require synchronisation, especially in shared memory systems.

Task Parallelism

- Focuses on distributing different tasks or functions across multiple processing units.
  - Each processing unit executes a different task, potentially on the same data.

- Characteristics
  - **Diverse Operations**: different tasks may perform different operations, possibly on the same data sets.
  - **Parallel Execution**: tasks are often independent and can be executed in parallel without waiting for other tasks to complete.
  - **Inter-Task Communication**: may require communication between tasks, especially if they are part of a workflow or pipeline.
- Example
  - image processing: loading image -> applying filter -> detecting edges

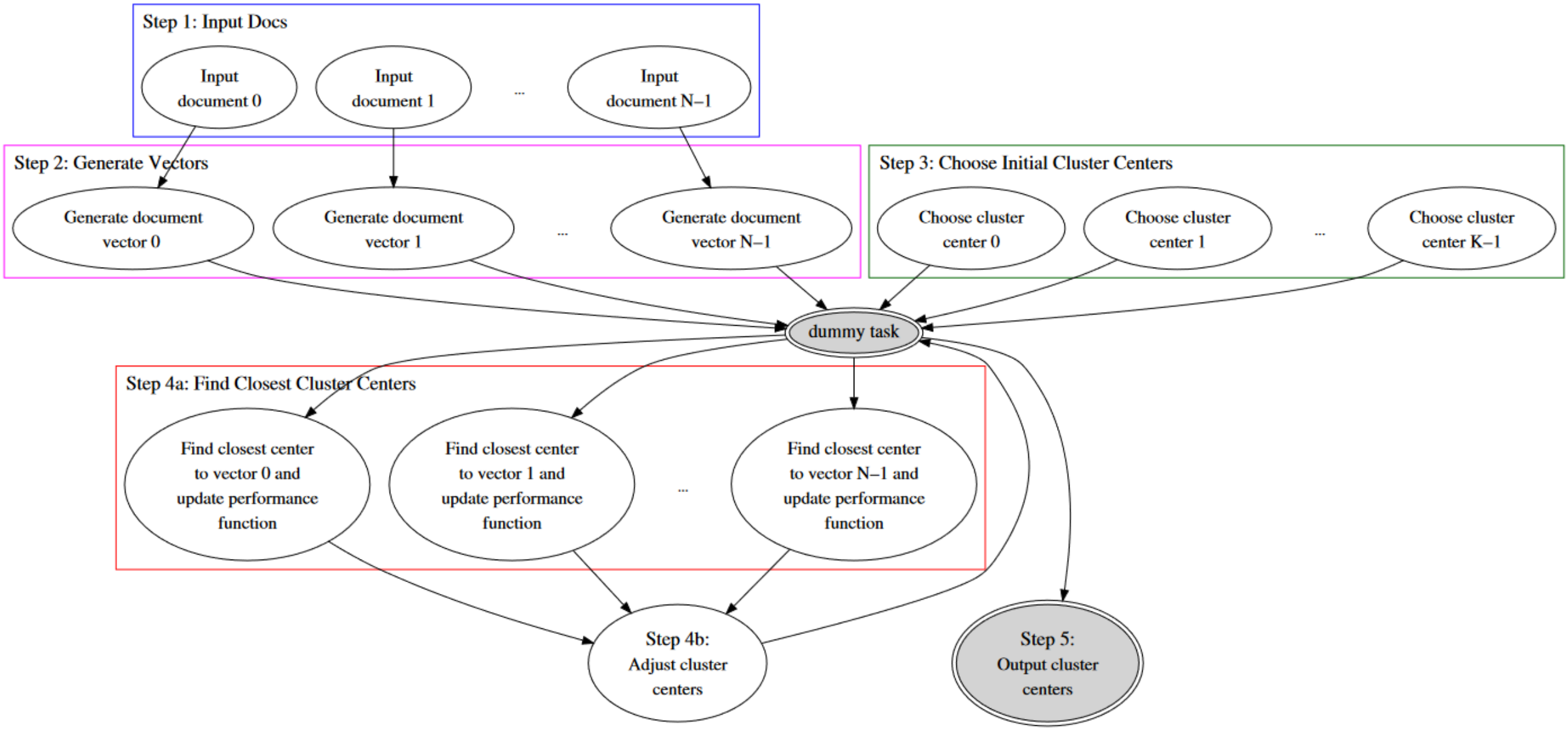
Example: Cluster Algorithm



```
1 Input N documents;
2 For each of the N documents generate a D-dimensional vector indicating how well it
      covers the D different topics;
3 Choose the K initial cluster centers using a random sample;
4 Repeat the following steps for I iterations or until the performance function
    converges, whichever comes first:
5       (a) For each of the N documents, find the closest center and compute its
            contribution to the performance function;
6       (b) Adjust the K cluster centers to try to improve the performance function
            value;
7 Output the K centers;
```

Steps for a Cluster algorithm, given is a collection of N text documents, with D topics and K possible clusters

src: https://www.cs.hunter.cuny.edu/~sweiss/course_materials/csci493.65/lecture_notes/chapter01.pdf



src: https://www.cs.hunter.cuny.edu/~sweiss/course_materials/csci493.65/lecture_notes/chapter01.pdf

Today, We Are Covering…

- What Are Parallelisable Problems?
- Their Characterisations
- Decomposition Analysis
- Data Parallelism vs. Task Parallelism
- Amdahl's Law and Scalability

Amdahl's Law and Scalability

- Amdahl's law: a formula to **find the (theoretical) maximum improvement in performance** of a system when only part of the system is improved.
  - i.e. speed up in latency (elapsed time between start and end of execution)
  - Particularly when adding more processors.

- The whole task **speedup increases with the improvement** of the resources of the system (e.g., adding more threads, cores, machines).
- But… regardless of the available improvement, the **max speedup is limited by the part of the system that cannot benefit from the improvement**.

Formula (1-2)

$$\text{Overall Speedup} = \frac{\text{Old execution time}}{\text{New execution time}}$$

$$= \frac{1}{\left(1 - \text{Fraction}_{\text{enhanced}} \right) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}}}$$

Non-speedup part (Serial Fraction)

Speedup factor (e.g., num of processors)

Speedup part (Parallel Fraction)

- **Diminishing Returns**: as more processors are added, the contribution of the parallel portion of the program to overall speedup decreases.
- Regardless of the available improvement (if $\text{Speedup}_{\text{enhanced}}$ is infinite), the **max speedup is limited by the part of the system that cannot benefit from the improvement (i.e. the Serial Fraction)**.

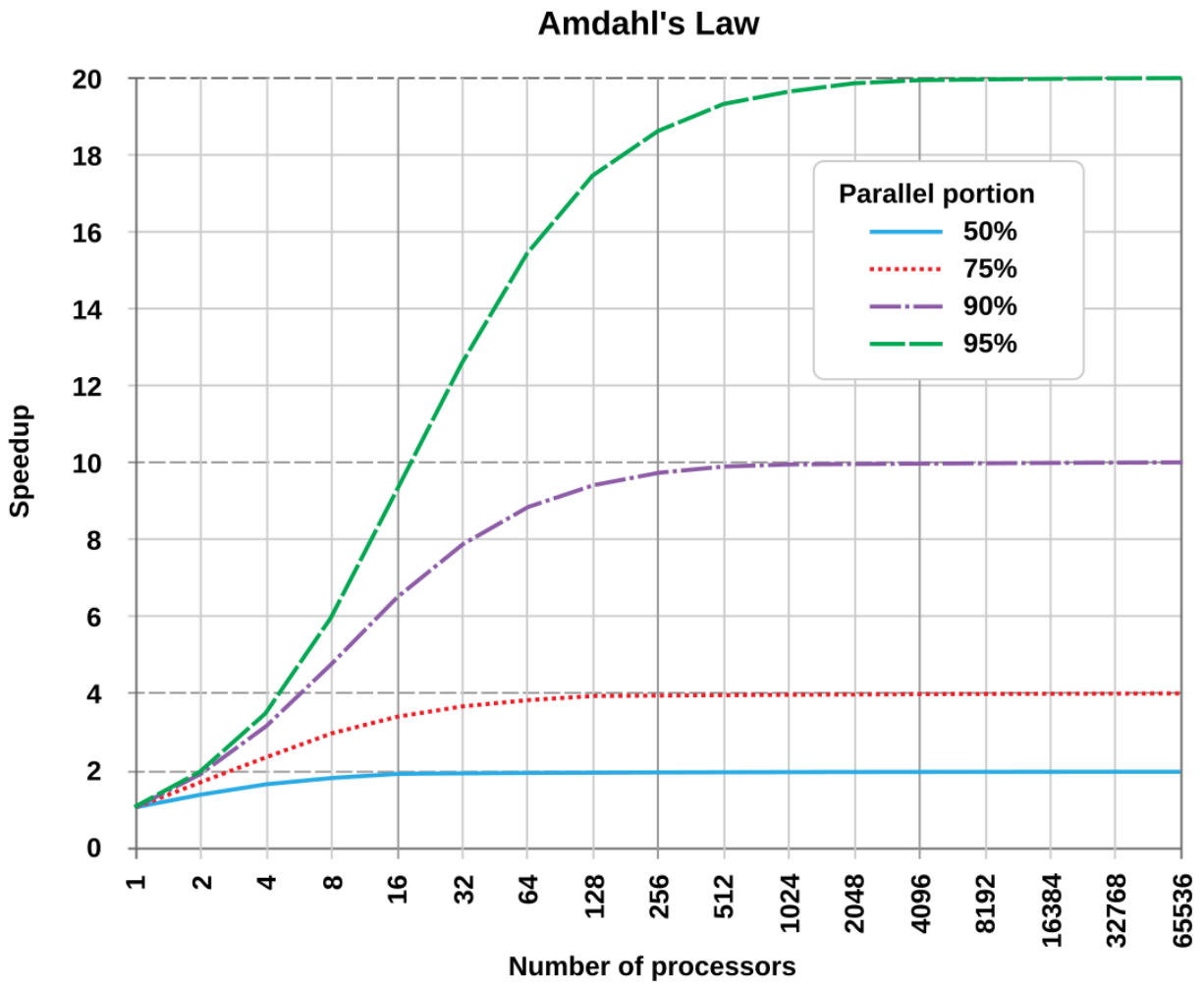
Formula (2-2)

$$Overall\ Speedup = S(f) = \frac{1}{((1-p) + {}^{p}/_{f})}$$

Where,
p: the portion of the system that can be parallelised
f: speedup factor (can be also the number of processors, N)

$$\boldsymbol{Max}\ Speedup = \lim_{f \to \infty} S(f) = \frac{1}{(1-p)}$$

Graphical Representation



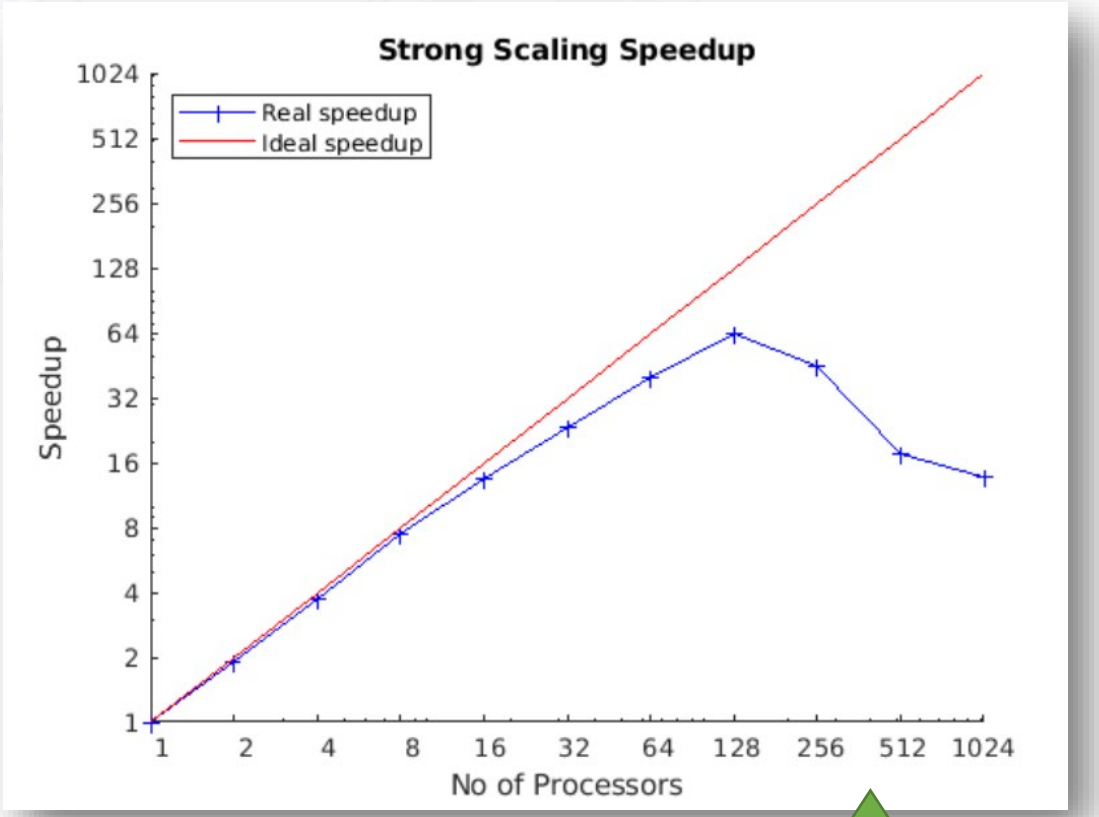
Example

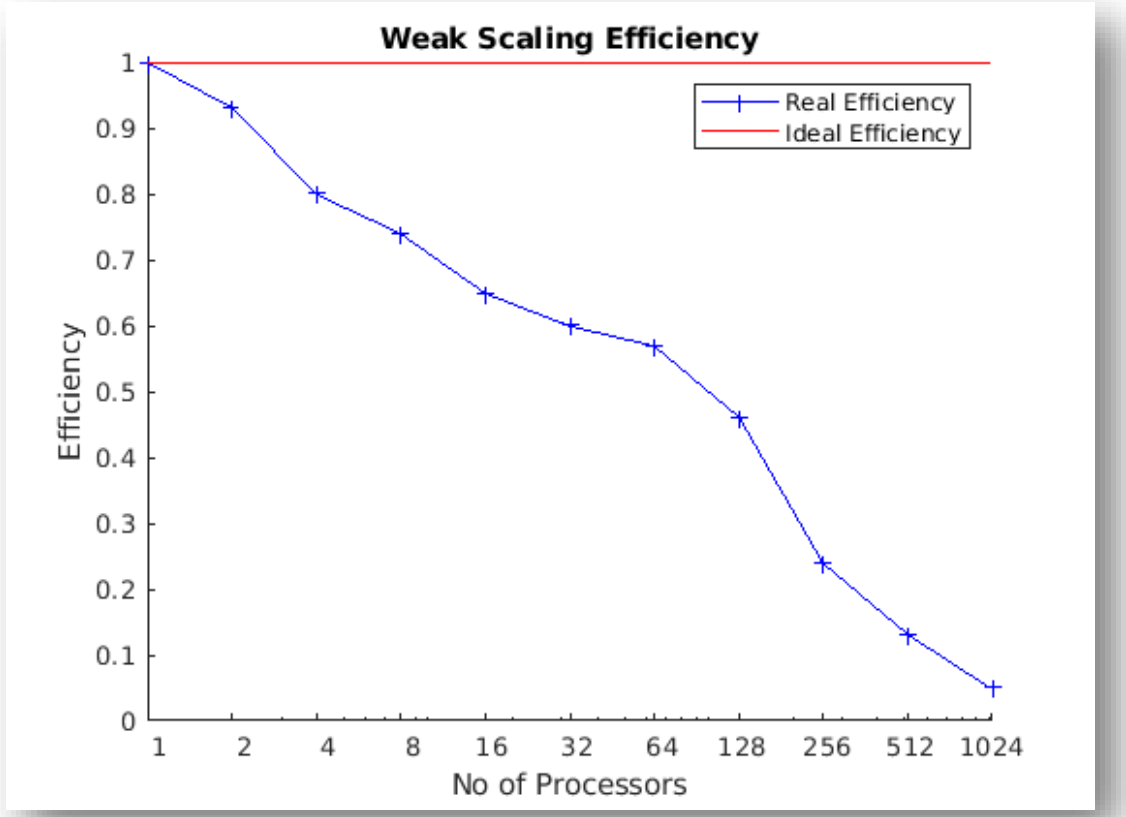
- Consider a program where 80% of the task is parallelisable (P = 0.8)
- Calculate speedup for factors (f or N): 2, 4, 8, 16
- Solution:
  - $S(2) = \frac{1}{((1-.8)+{}^{.8}/_{2})} = 1.67$
  - $S(4) = \frac{1}{((1-.8)+{}^{.8}/_{4})} = 2.5$
  - $S(8) = \frac{1}{((1-.8)+{}^{.8}/_{8})} = 3.33$
  - $S(16) = \frac{1}{((1-.8)+{}^{.8}/_{16})} = 4$

Scalability

- Reflects how **effectively a system can utilise additional computational resources** (like processors) to solve a problem more quickly or solve a larger problem within the same time.
- Amdahl's Law shows that **scalability is fundamentally limited by the serial portion** of the program.
- Strong vs. Weak Scaling:
  - **Strong Scaling**: increasing the number of processors for a fixed problem size.
    - The goal is to reduce execution time of the problem by adding more processors.
  - **Weak Scaling**: increasing the number of processors while increasing the problem size proportionally.
    - The goal is to maintain a constant execution time.

Strong vs Weak Scalability





Amdahl is applied to this
fixed problem size & increased resources

src: https://hpc-wiki.info/hpc/Scaling#Strong_Scaling_2

Strong vs Weak Scaling

Aspect	Strong Scaling	Weak Scaling
Problem Size	Fixed	Increases proportionally with the number of processors
Objective	Reduce execution time	Maintain constant execution time
Speedup Measurement	How execution time decreases with more processors	How execution time remains constant with increased problem size and processors
Ideal Scenario	Linear speedup	Constant execution time
Practical Challenges	Diminishing returns due to overheads	Managing increased communication and synchronization overhead as problem size grows
When to Use	When the problem size is fixed and reducing time is the goal	When dealing with large datasets that can be split across more processors

LERO Software for a better world

A World Leading SFI Research Centre

Science Foundation Ireland sfi For what's next

What Are Your Questions?

HOST INSTITUTION
UNIVERSITY OF LIMERICK OLLSCOIL LUIMNIGH

PARTNER INSTITUTIONS
NUI Galway OÉ Gaillimh
Maynooth University
Trinity College Dublin Coláiste na Tríonóide, Baile Átha Cliath The University of Dublin
University College Dublin An Coláiste Ollscoile, Baile Átha Cliath
UCC
MTU Ollscoil Teicneolaíochta na Mumhan Munster Technological University
DCU
Waterford Institute of Technology
Dundalk Institute of Technology Dhún Dealgan
LIT
GMIT